PORTFOLIO

서비스가 멈추지 않는 인프라를 설계 하는 엔지니어 오지민 입니다.

Github
wlals2

blog
jiminhome.shop

Phone
010-8278-3213

Email
fw4568@gmail.com

프로젝트 개요

아마존 동물병원 : 24/7 프랜차이즈 동물병원 예약 플랫폼

가맹 2,500+개 | 월 100만 예약 | 일 3만 트래픽

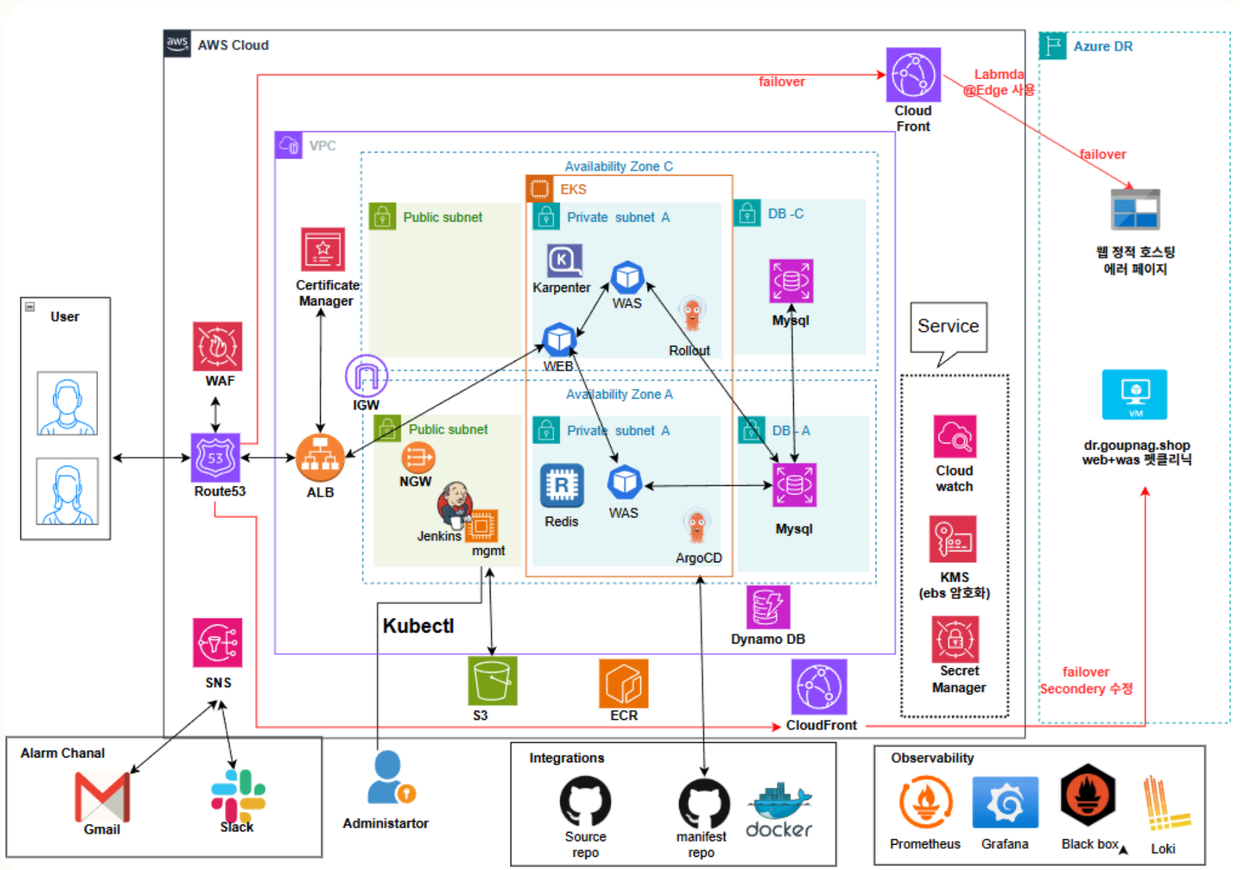
역할 : 4인팀 팀장 / 전체 아키텍처 설계 + Terraform 코드 구축

고객 요구 사항

- 서비스 연속성 → Multi-Cloud DR
- 트래픽 변동 → EKS + HPA + Karpenter
- 무중단 배포 → ArgoCD + Canary(Argo Rollout)
- 인프라 일관성 및 관리 → Terraform
- 모니터링 → CloudWatch + Prometheus + Grafana + Loki

핵심 성과

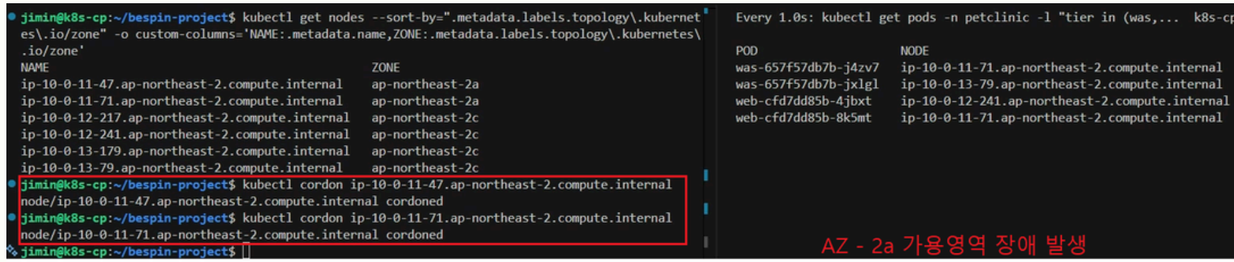
- DR System: 고객 요구 RTO 6시간 → 실제 30분 달성
- DR 평상시 비용 $10 미만 (Cold stand by 설계)
- 큰 트래픽 변동 사항에도 문제 없는 HA 설정
- 모니터링 시스템으로 빠르게 장애 대응

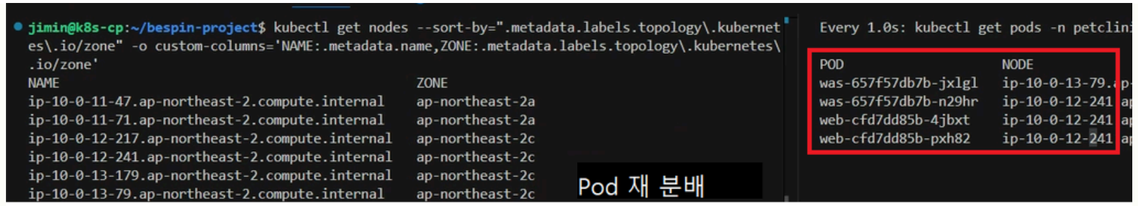


고가용성 EKS 아키텍처

설계 목표 : 24/7 동물 병원 예약 플랫폼 → 단일 장애 지점 (SPOF) 제거

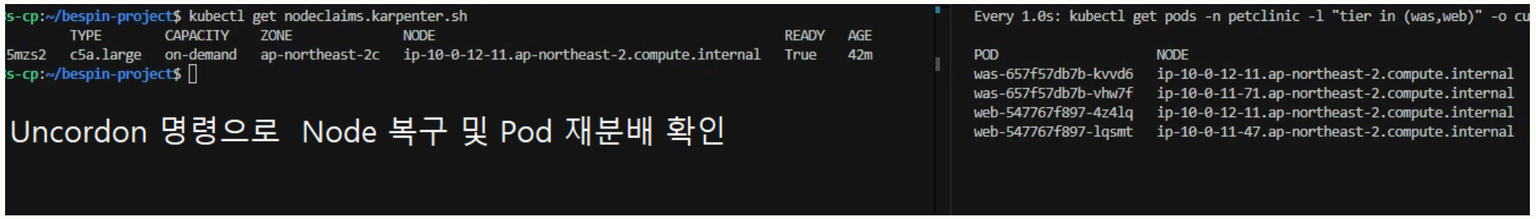
- **Multi-AZ 배치**
  - ap-northeast-2a + 2c
  - **하나의 AZ 장애 시에도 서비스유지**
- **Pod 분산 전략**
  - **Pod Anti-Affinity** → 노드 부족시 Pod 스케줄 불가 **(탈락)**
  - **ToplogySpreadConstraints + ScheduleAnyway** → AZ 균등 분산 시도 불가능하면 어디든 배치 **(채택)**
- **오토스케일링**
  - **HPA (Pod):** CPU 80%, min2 ,max 10 **→ 트래픽 증가 시 Pod 확장**
  - **Karpenter(Node):** 워크 로드 기반 자동 프로비저닝 → Pod Pending 시 노드 추가
  - **Karpenter 운영 오버헤드 문제로 Spot 삭제 On-demand 사용**





- **시연: AZ 장애 + 대형 트래픽 동시 발생**
  - AZ-A Cordon (장애 시뮬레이션)
  - Replica 10 (대형 트래픽 발생)

  --> AZ-C로 Pod 자동 분산 + Karpenter 노드추가
  - uncordon 노드 복구 + Pod 재 분배



- **성과 : 단일 장애 지점(SPOF) 제거 — AZ 장애 시에도 고가용성 서비스 유지**

DR 아키텍처

팀 내 논의 : Warm vs Cold 비교

- **Warm standby :** AKS + DMS 실시간 동기화 → 높은 복구 속도, 똑같은 컨테이너 환경
- **Cold Standby**: Azure VM + mysqldump 백업 복원 → 비용 최적화, Terraform 인프라 구성

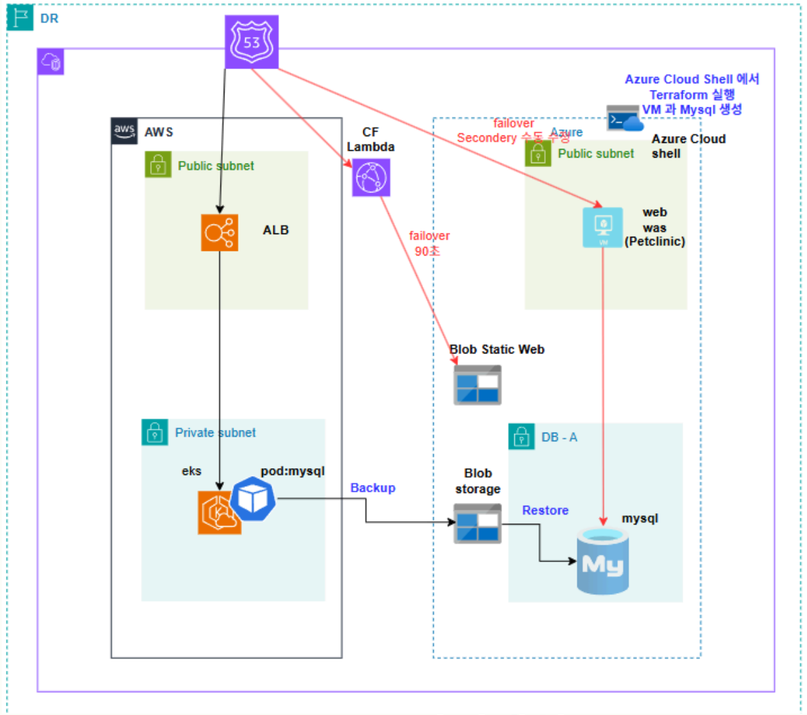
선택: Cold standby

- 금융 기관이 아닌 동물 병원 — 수 분 복구 불필요
- AWS 리전 장애는 일년에 자주 일어나지 않음
- Terraform 코드가 있어 빠르게 프로비저닝 가능 **(13분)**

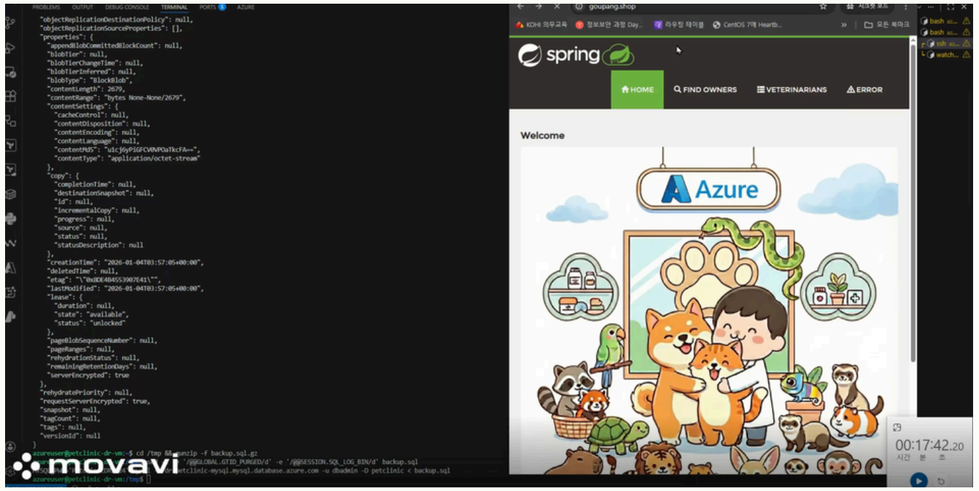
DR 복구 흐름 → RTO 30분 달성

- CloudFront 점검 페이지 자동 전환 (~3분)
- Terraform으로 Azure VM 프로비저닝(~13분)
- mysqldump 백업 에서 DB 복원 (~2분)
- Route 53 DNS 수동 전환 (~10분)

고객 요구 RTO 6시간 → 30분 이내 복구 완료



DR 아키텍처



DR 시연 — Azure 17분 서비스 복구

CI/CD & Canary 배포 — Redis 세션 관리

CI/CD 파이프라인 & 무중단 배포

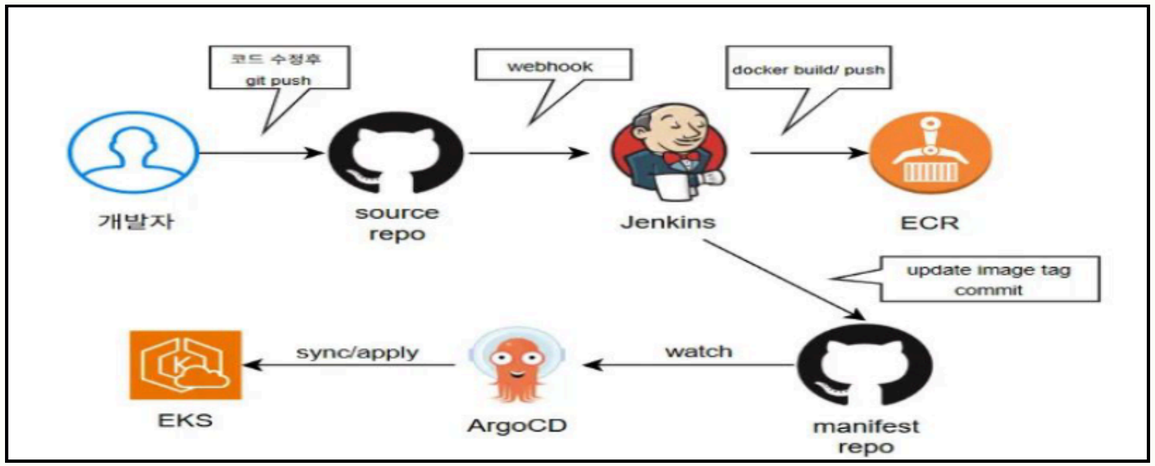
- **Jenkins(빌드) → ECR(이미지)-> ArgoCD(배포) → EKS**
- **"빌드와 배포를 분리"** Jenkins는 코드만, ArgoCD는 배포만
- **Canary 배포** → 무중단 배포 가능
- **ALB TrafficRouting** → TG 트래픽을 이용한 가중치 분배 가능

문제: Canary 배포 에서 세션이 깨짐 - 세션 Trade off

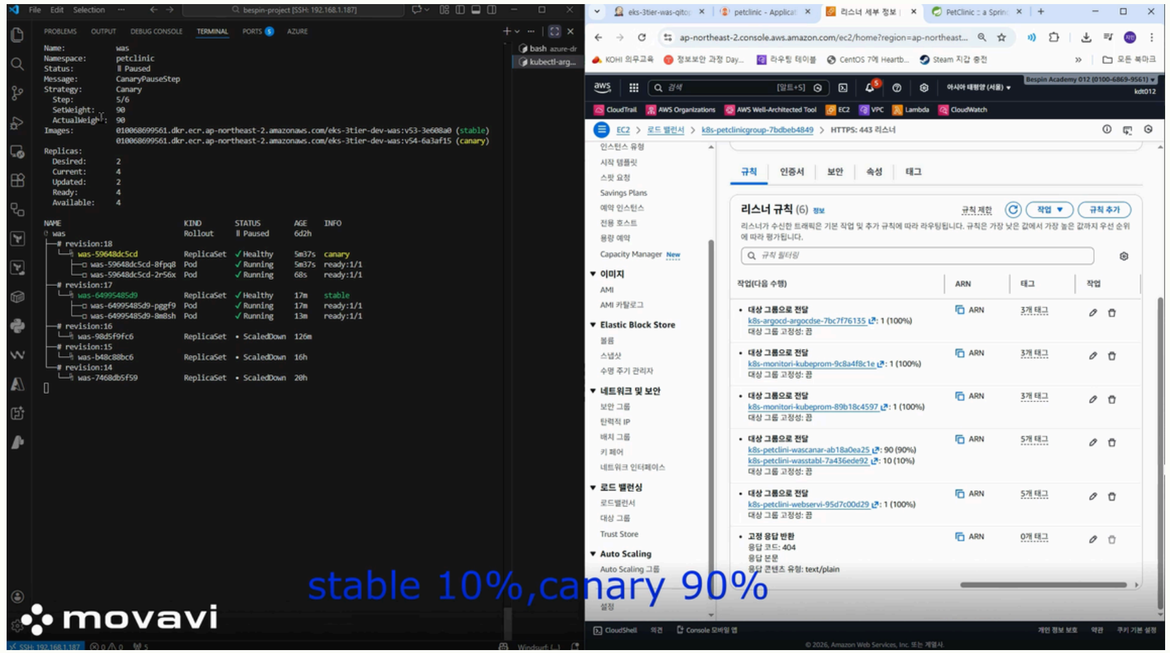
- **Sticky Session** : Canary 특성상 Tartget Group 정보 공유 불가
- **JWT** : 웹 개발 오버 헤드
- **Redis** :최소 개발 요구사항

Redis 채택: Spring Session Redis 의존성 추가만으로 세션 외부화 완료

결과: Canary 배포시 세션 유지 + 무중단 배포 달성



CI/CD 파이프라인



카나리 배포 확인

모니터링 시스템

모니터링 아키텍처

- **CloudWatch** : ALB, Route53, EKS 등 리소스 메트릭
- **PLG Stack**: K8s 커스텀 메트릭 수집
- **Grafana 대쉬보드 구축** (Session, RDS backup, Karpetner 등)
- **Loki** : 로그 중앙화 를 통해 장애시 빠르게 확인 가능

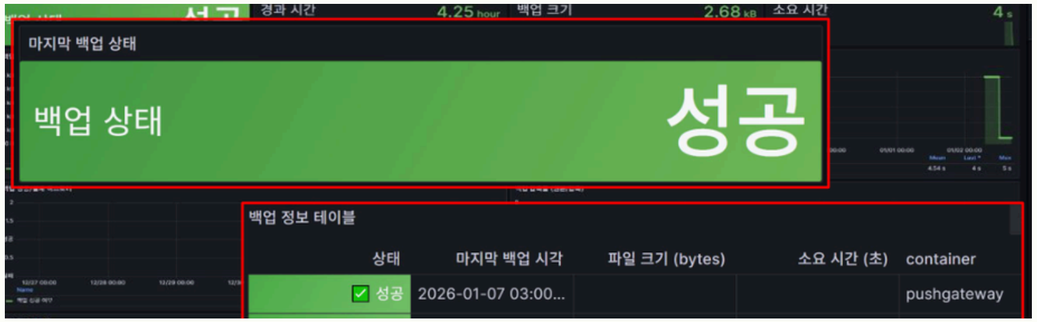
성과 : CloudWatch 수집 **메트릭 비용 최적화 ,** 커스텀 알림 제작(jenkins 빌드 알람)

알람 — 위험도 3단계 분류 + 대응 시간 정의

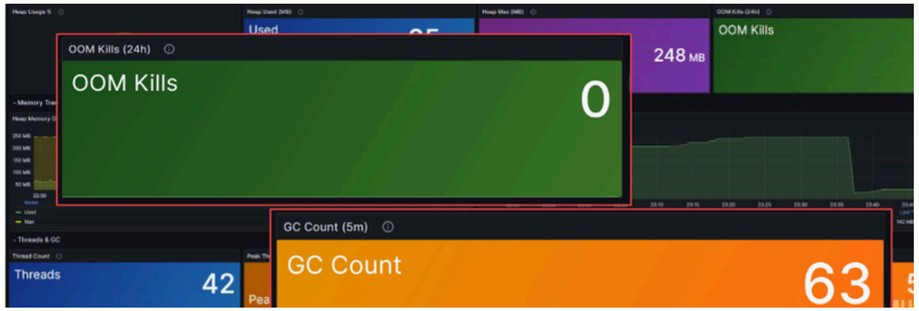
- **Critical (5분 이내)** : 노드 장애, RDS, Route53 Failover 등
- **High (30분 이내)**: 메모리 초과, 5XX 에러, DB 커넥션 등
- **Medium (수 시간내)**: CPU 사용률 , 지연시간 초과 등

실제 경험

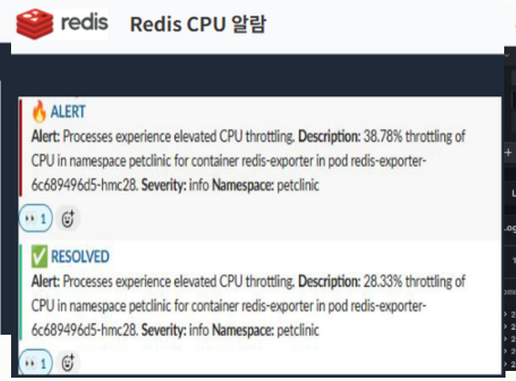
- **Redis CPU throttling 38.78%** → 장애 전 감지 + 대응 완료
- **백업 CronJob 미동작** → Push Gateway 에서 발견 및 에러 수정



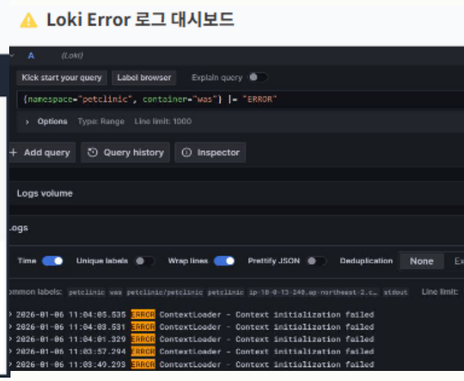
RDS 백업 대쉬보드



애플리케이션 대쉬보드



알람



Loki 로그 확인

트러블 슈팅 및 회고

- **Terraform State 삭제사고**
  - **증상:** State 파일 삭제
    - 인프라 실제 상태와 코드 불일치
    - Terraform이 기존 리소스 재생성 시도
  - **해결:** 팀원 간 코드 수정 통합
  - **재발 방지:** DynamoDB Lock (동시 수정차단) + S3 State 백업

- **CloudFront → Azure Blob 400 Bad Request**
  - **증상:** DR 점검 페이지 접속 시 400 에러
  - **원인:** Azure Blob Host 헤더 검증 + Cloud Front가 원래 Host 전달
  - **해결:** Lambda@Edge로 Host 변경 + 동작하나 복잡도 + 비용 증가
  - **고도화:** Azure Static Web Apps 교체 (무료 + HTTPS 자동)

- **ArgoCD + HPA 충돌**
  - **증상:** HPA → Pod 5개 확장 → ArgoCD Git(2개로) 되돌림 → 무한반복
  - **원인:** ArgoCD가 /spec/replicas drift 감지 → 강제 동기화
  - **해결:** ignoreDifferences로 replicas 필드를 감시에서 제외

- **HTTPS Redirect Loop**
  - **증상:** https://www.goupang/petclinic/ → “Too many redirects”
  - **원인:** ALB(HTTPS) → Nginx (HTTP), nginx 가 X-Forwarded-Proto 확인
  - **해결**: nginx.conf 에서 X-Forwarded-Proto 헤더 확인 로직 추가

"서비스가 멈추지 않는 인프라를 설계하는 엔지니어 오지민 입니다."

장애가 발생해도 서비스 지속하는 인프라, 조기에 장애를 감지하여 대응 할 수있는 시스템을 만들어 회사에 기여하겠습니다.